SmartLibrary — Cross-Module Project

Group Name:

Module: Cross-Module Project (Smart Library Application)

Deliverables included in this document:

1. Mock interview transcript with Library Manager
2. Requirements Specification — Actors, Use Cases, Functional & Non-Functional requirements
3. Wireframes / sketches (textual + simple diagrams)
4. Database design (ERD, PostgreSQL DDL)
5. Backend (Python, OOP) — classes, business rules, and full CRUD using psycopg2 (DAOs/repositories)
6. PyQt5 desktop GUI skeleton and example windows (login, catalog, loan workflow, book club management)
7. Test plan and sample test cases
8. Presentation plan and speaker notes (PowerPoint slide-by-slide outline)
9. One-page summary (for upload to Canvas)

1. Mock Interview with Library Manager

Interviewer (Student): Good morning — thank you for meeting with us. Can you tell us about the library's main responsibilities and pain points?

Library Manager: The library serves students and staff. We manage book loans, membership, book clubs, and occasional events. Main pain points are manual record keeping, slow search for books, overdue tracking, and difficulty managing multiple copies and reservations.

Interviewer: How do users currently borrow and return books?

Library Manager: They fill a paper form at the front desk. Staff manually write member ID and book IDs. There's no automatic overdue notification or limits enforcement except memory.

Interviewer: What reports would be useful?

Library Manager: Daily loans summary, overdue list, most-borrowed books, active members, and book club attendance.

Interviewer: Any special rules or policies we must enforce?

Library Manager: Max 3 active loans per member. Loan period 7 days. Staff (librarian) can override fines or extend loans. Members must authenticate with library ID and PIN.

Interviewer: Anything else?

Library Manager: Book club admins should be able to create and manage clubs and memberships, and generate reading lists.

Interviewer: Thank you, that helps shape requirements.

2. Requirements Specification

2.1 Overview

SmartLibrary is a desktop application that supports Limkokwing University central library. It will provide CRUD management for books, authors, members, loans, and book clubs, enforce business rules (max 3 loans, 7-day due), and include role-based authentication for Librarian and Member users.

2.2 Actors

- **Librarian** — administrative user: manage inventory, members, loans, book clubs, reports.
- **Member** — library patron: search/catalog, request/return books, join clubs.
- **System Administrator (optional)** — manage system-level configuration and backups.

2.3 Use Cases (high level)

1. **Authenticate** (login) — input username/PIN, return role-based view.
2. **Manage Books** (CRUD) — add/edit/delete books and copies, search/filter books.
3. **Manage Authors** (CRUD) — add/edit author records, link authors to books.
4. **Manage Members** (CRUD) — register member, update details, disable.
5. **Loan Book** — create loan if member loans < 3 and copies available; set due date = loan_date + 7 days.
6. **Return Book** — close loan record, calculate overdue if any.
7. **Book Club Management** — create/modify book clubs, add/remove members, schedule meetings.
8. **Reports & Dashboard** — most-borrowed books, overdue loans, active clubs.
9. **Search & Filter** — full-text or attribute search on title, author, ISBN, category.

10. **Role-based Access Control** — Librarian vs Member actions.

2.4 Functional Requirements

Books & Authors - FR1: System shall provide CRUD operations for Book entities. - FR2: System shall allow multiple copies for a Book (track `total_copies`, `available_copies`). - FR3: System shall allow searching books by title, author, ISBN, and category.

Members & Authentication - FR4: System shall provide CRUD for members and store `member_id`, `name`, `email`, `phone`, `pin_hash`, `role`. - FR5: System shall authenticate users with username and PIN (hashed in DB).

Loans - FR6: System shall create a Loan record when a librarian issues a book. - FR7: Business rule: maximum 3 active loans per member. - FR8: Loan due date shall be 7 days from loan date and stored on Loan. - FR9: System shall mark loan `returned_at` when returned and update `available_copies`.

Book Clubs - FR10: System shall allow CRUD for BookClub entity and membership linking. - FR11: System shall record BookClub meetings and attendees.

Reports & Dashboard - FR12: System shall produce a dashboard summary: top 10 borrowed books, overdue count, active clubs. - FR13: System shall export report as CSV.

Integration - FR14: Use PostgreSQL for persistence; use psycopg2 for DB connectivity.

2.5 Non-Functional Requirements

- NFR1 (Performance): Search queries should return results within 2 seconds for 10k records on modest hardware.
- NFR2 (Security): PINs must be hashed (bcrypt). Database connections must use secure credentials; do not store raw credentials in code.
- NFR3 (Usability): GUI must be role-aware and minimize clicks for common flows (issue/return).
- NFR4 (Reliability): Data must preserve ACID properties; use transactions for loan operations.
- NFR5 (Maintainability): Code must use OOP, be modular, and include unit tests for DAOs and business rules.

2.6 Wireframes / Sketches (textual)

- **Login screen:** fields: Username, PIN; buttons: Login, Forgot PIN.
- **Dashboard (Librarian):** KPIs at top; quick actions: Add Book, Register Member, Issue Book; tables: Recent Loans, Overdue Loans.
- **Book Catalog screen:** Search bar, filters (author, category), results table with `Issue` button per row.
- **Loan screen (modal):** choose member (search), choose copy, confirm issue.
- **Book Club screen:** list clubs, create club button, manage members.

3. Database Design (PostgreSQL)

3.1 ERD (textual)

Entities: author, book, book_copy, member, "user" (for auth), loan, book_club, book_club_member. Relationships: - book – many-to-many –> author (via book_author) - book – 1-to-many –> book_copy - member – 1-to-many –> loan - book_club – many-to-many –> member (via book_club_member)

3.2 PostgreSQL DDL (schema)

```
-- PostgreSQL schema for SmartLibrary
CREATE TABLE author (
  id SERIAL PRIMARY KEY,
  full_name TEXT NOT NULL,
  biography TEXT
);

CREATE TABLE book (
  id SERIAL PRIMARY KEY,
  title TEXT NOT NULL,
  isbn TEXT UNIQUE,
  category TEXT,
  description TEXT,
  total_copies INT DEFAULT 1
);

CREATE TABLE book_author (
  book_id INT REFERENCES book(id) ON DELETE CASCADE,
  author_id INT REFERENCES author(id) ON DELETE CASCADE,
  PRIMARY KEY (book_id, author_id)
);

CREATE TABLE book_copy (
  id SERIAL PRIMARY KEY,
  book_id INT REFERENCES book(id) ON DELETE CASCADE,
  copy_number INT NOT NULL,
  status TEXT DEFAULT 'available' -- available, loaned, damaged
);

CREATE TABLE member (
  id SERIAL PRIMARY KEY,
  library_id VARCHAR(20) UNIQUE NOT NULL,
  full_name TEXT NOT NULL,
  email TEXT,
  phone TEXT,
  pin_hash TEXT NOT NULL,
```

```
  role TEXT DEFAULT 'member' -- member or librarian
);

CREATE TABLE loan (
  id SERIAL PRIMARY KEY,
  book_copy_id INT REFERENCES book_copy(id),
  member_id INT REFERENCES member(id),
  loaned_at TIMESTAMP WITH TIME ZONE DEFAULT now(),
  due_at TIMESTAMP WITH TIME ZONE NOT NULL,
  returned_at TIMESTAMP WITH TIME ZONE
);

CREATE TABLE book_club (
  id SERIAL PRIMARY KEY,
  name TEXT NOT NULL,
  description TEXT,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT now()
);

CREATE TABLE book_club_member (
  club_id INT REFERENCES book_club(id) ON DELETE CASCADE,
  member_id INT REFERENCES member(id) ON DELETE CASCADE,
  joined_at TIMESTAMP WITH TIME ZONE DEFAULT now(),
  PRIMARY KEY (club_id, member_id)
);

-- Indexes for search
CREATE INDEX idx_book_title ON book USING gin (to_tsvector('english', title))
;
```

Notes: `available_copies` can be computed as `total_copies - count(loan where returned_at is null)` or tracked by triggers if preferred.

4. Python Backend (OOP) — Overview and Code Snippets

4.1 Structure

```
smartlibrary/
  db/
    connection.py
    dao_author.py
    dao_book.py
    dao_member.py
    dao_loan.py
  models/
    author.py
    book.py
    book_copy.py
```

```
    member.py
    loan.py
  services/
    loan_service.py
  gui/
    main.py (PyQt5 entry)
  utils/
    security.py (bcrypt wrapper)
```

4.2 Key model class examples

```python
# models/member.py
from dataclasses import dataclass

@dataclass
class Member:
    id: int | None
    library_id: str
    full_name: str
    email: str | None
    phone: str | None
    pin_hash: str
    role: str = 'member'

    @property
    def is_librarian(self):
        return self.role == 'librarian'
```

```python
# models/book.py
from dataclasses import dataclass

@dataclass
class Book:
    id: int | None
    title: str
    isbn: str | None
    category: str | None
    description: str | None
    total_copies: int = 1
```

4.3 Database connection (db/connection.py)

```python
import psycopg2
from psycopg2.extras import RealDictCursor
import os

def get_conn():
    return psycopg2.connect(
        dbname=os.getenv('DB_NAME','smartlib'),
        user=os.getenv('DB_USER','postgres'),
        password=os.getenv('DB_PASS','password'),
```

```
        host=os.getenv('DB_HOST','localhost'),
        port=os.getenv('DB_PORT', '5432')
    )
```

4.4 Example DAO (dao_loan.py) with business rule enforcement

```
# dao_loan.py
from db.connection import get_conn
from datetime import timedelta, datetime

MAX_LOANS = 3
LOAN_DAYS = 7

class LoanDAO:
    def create_loan(self, member_id:int, book_copy_id:int):
        conn = get_conn()
        try:
            with conn:
                with conn.cursor() as cur:
                    # check active loans
                    cur.execute("SELECT count(*) FROM loan WHERE member_id=%s
AND returned_at IS NULL", (member_id,))
                    (active_count,) = cur.fetchone()
                    if active_count >= MAX_LOANS:
                        raise Exception('Member has reached max active loans'
)

                    # check copy availability
                    cur.execute("SELECT status FROM book_copy WHERE id=%s FOR
UPDATE", (book_copy_id,))
                    row = cur.fetchone()
                    if not row or row[0] != 'available':
                        raise Exception('Copy not available')

                    due = datetime.utcnow() + timedelta(days=LOAN_DAYS)
                    cur.execute(
                        "INSERT INTO loan (book_copy_id, member_id, due_at) V
ALUES (%s,%s,%s) RETURNING id",
                        (book_copy_id, member_id, due)
                    )
                    loan_id = cur.fetchone()[0]
                    cur.execute("UPDATE book_copy SET status='loaned' WHERE i
d=%s", (book_copy_id,))
                    return loan_id
        finally:
            conn.close()
```

4.5 Return flow

```
def return_loan(self, loan_id:int):
    conn = get_conn()
    try:
        with conn:
            with conn.cursor() as cur:
                cur.execute("SELECT book_copy_id, returned_at FROM loan WHERE
id=%s FOR UPDATE", (loan_id,))
                row = cur.fetchone()
                if not row:
                    raise Exception('Loan not found')
                if row[1] is not None:
                    raise Exception('Already returned')
                book_copy_id = row[0]
                cur.execute("UPDATE loan SET returned_at=now() WHERE id=%s",
(loan_id,))
                cur.execute("UPDATE book_copy SET status='available' WHERE id
=%s", (book_copy_id,))
    finally:
        conn.close()
```

Security note: Use prepared statements and parameterized queries as shown to avoid SQL injection.

5. PyQt5 GUI (skeleton)

5.1 Main entry (gui/main.py)

```
import sys
from PyQt5.QtWidgets import QApplication, QMainWindow
from gui.login import LoginWindow

if __name__ == '__main__':
    app = QApplication(sys.argv)
    login = LoginWindow()
    login.show()
    sys.exit(app.exec_())
```

5.2 Example Login window (gui/login.py)

```
from PyQt5.QtWidgets import QWidget, QLineEdit, QPushButton, QVBoxLayout, QLa
bel, QMessageBox
from db.connection import get_conn
from utils.security import verify_pin

class LoginWindow(QWidget):
    def __init__(self):
        super().__init__()
```

```
        self.setWindowTitle('SmartLibrary - Login')
        layout = QVBoxLayout()
        layout.addWidget(QLabel('Library ID'))
        self.user = QLineEdit()
        layout.addWidget(self.user)
        layout.addWidget(QLabel('PIN'))
        self.pin = QLineEdit()
        self.pin.setEchoMode(QLineEdit.Password)
        layout.addWidget(self.pin)
        btn = QPushButton('Login')
        btn.clicked.connect(self.handle_login)
        layout.addWidget(btn)
        self.setLayout(layout)

    def handle_login(self):
        library_id = self.user.text().strip()
        pin = self.pin.text().strip()
        # fetch user
        conn = get_conn()
        with conn:
            with conn.cursor() as cur:
                cur.execute("SELECT id, pin_hash, role FROM member WHERE libr
ary_id=%s", (library_id,))
                row = cur.fetchone()
                if not row:
                    QMessageBox.warning(self, 'Login failed', 'Invalid creden
tials')
                    return
                member_id, pin_hash, role = row
                if not verify_pin(pin, pin_hash):
                    QMessageBox.warning(self, 'Login failed', 'Invalid creden
tials')
                    return
                # open main dashboard window depending on role
                QMessageBox.information(self, 'Login', f'Welcome {role}')
```

5.3 Book Catalog screen

- Provide table with search bar, filter dropdowns, and `Issue` button. Implement with `QTableView` bound to a model.

6. Security Utilities

```
# utils/security.py
import bcrypt

def hash_pin(pin: str) -> str:
    return bcrypt.hashpw(pin.encode('utf-8'), bcrypt.gensalt()).decode('utf-8
')
```

```
def verify_pin(pin: str, hashed: str) -> bool:
    return bcrypt.checkpw(pin.encode('utf-8'), hashed.encode('utf-8'))
```

Note: add `bcrypt` to your `requirements.txt`.

7. Testing and Test Cases

Unit tests (eg.): -

Test creating a loan: member with 0 active loans should be able to borrow; after 3 loans, next loan fails. - Test return flow: returned_at is set and book_copy status restored. - Test authentication: correct PIN accepted, wrong PIN rejected.

Integration tests: simulate issuing a loan and verify DB state changes in a transaction that is rolled back.

8. Presentation (PowerPoint) Outline and Speaker Notes

Slide 1 — Title: SmartLibrary — Team names, Module, Date. **Slide 2 — Problem Statement**: Current manual processes & pain points. **Slide 3 — Objectives**: What the system will achieve. **Slide 4 — Actors & Use Cases**: quick bullets and diagram. **Slide 5 — Database Design**: ERD (image or ASCII), key tables. **Slide 6 — Backend Architecture**: OOP, DAOs, business rules. **Slide 7 — GUI Screens**: Login, Catalog, Loan flow, Book Club. **Slide 8 — Demo Plan**: Steps to demonstrate issue/return and club management. **Slide 9 — Non-Functional Requirements & Security**. **Slide 10 — Testing & Next Steps**. **Slide 11 — Q&A**.

9 — One-Page Summary (for Canvas upload)

SmartLibrary — Desktop application for Limkokwing University central library to manage books, authors, members, loans, and book clubs. Provides role-based authentication, enforces business rules (max 3 loans, 7-day loan period), and features search, reports,

and club management. Implemented using Python (OOP), PostgreSQL, psycopg2, and PyQt5.

Key deliverables: Requirements spec, DB schema, Python backend with DAO implementations, PyQt5 GUI skeleton, sample test data, and presentation script.

10 — How to run (developer instructions)

1. Install PostgreSQL and create database `smartlib`.
2. Run the provided SQL DDL to create schema.
3. Create a `.env` or environment variables for DB credentials.
4. `pip install -r requirements.txt` (requirements: psycopg2-binary, PyQt5, bcrypt)
5. Run `python gui/main.py` to start the app.

11 Our Next steps (recommended deliverables to prepare outside this document)

- Populate sample seed data SQL for authors, books, copies, and a test member.
- Create unit tests (`pytest`) and integration test scripts.
- Produce the PowerPoint file (PPTX) from the slide outline — this document contains slide content; I can generate the PPTX if you want.
- Create a short demo video script and run-through.

End of document.